

Documentación: Patrón MVC + Observer

Space Invaders — Proyecto de Programación

Fecha: 16 de marzo de 2026

Índice

1. Introducción al Patrón MVC + Observer	2
2. Implementación en Space Invaders	2
2.1. Arquitectura MVC del proyecto	2
2.2. Diagramas UML	4
3. El Controlador en Detalle	4
3.1. Implementación del Controlador en Space Invaders	4
4. Patrón Observer: Notificación y Actualización	5
4.1. Método <code>notify</code>	6
4.2. Método <code>update</code>	6
4.3. Funcionamiento conjunto de <code>notify</code> y <code>update</code>	6
4.4. Aplicación al proyecto Space Invaders	7
4.4.1. Quién notifica y cuándo	7
4.4.2. Cómo responde <code>MainFrame</code> en su <code>update</code>	7
4.4.3. Cómo responde <code>StartFrame</code> en su <code>update</code>	8
5. Conclusiones	8

Introducción al Patrón MVC + Observer

El patrón **Modelo-Vista-Controlador** (MVC) es una arquitectura de diseño utilizada en programación orientada a objetos que permite organizar una aplicación separando sus componentes en tres partes principales: *Modelo*, *Vista* y *Controlador*. Esta separación facilita el mantenimiento del código, mejora su organización y permite que cada componente tenga una responsabilidad clara dentro del sistema.

Modelo

Se encarga de gestionar los datos y la lógica del programa. Contiene toda la lógica de negocio, del juego en este caso, y las operaciones necesarias para modificarla o consultarla. El modelo no depende de la interfaz gráfica ni de cómo se muestran los datos al usuario.

Vista

Es la parte encargada de presentar la información al usuario. Su función principal es mostrar los datos que proporciona el modelo y actualizarse cuando estos cambian. La vista no contiene la lógica principal del programa, sino únicamente la representación visual de la información.

Controlador

Actúa como intermediario entre el usuario y el sistema. Recibe las acciones del usuario (por ejemplo, clics, entradas de teclado, etc.), interpreta estas acciones y solicita al modelo que realice las operaciones correspondientes.

A este modelo de arquitectura se le añade el patrón **Observer** (**Observador**) para que la vista se mantenga actualizada cuando cambian los datos del modelo. Este **patrón de diseño** establece una relación entre un objeto denominado *observable* (el modelo) y uno o varios *observadores* (las vistas). Cuando el estado del observable cambia, todos los observadores registrados son notificados automáticamente y pueden actualizar su información.

La combinación de MVC y Observer permite desarrollar aplicaciones más modulares, donde la lógica del programa, la presentación de los datos y la interacción con el usuario están claramente separadas. Esto facilita la reutilización del código, la ampliación del sistema y el mantenimiento a largo plazo de la aplicación.

Implementación en Space Invaders

Arquitectura MVC del proyecto

En el proyecto Space Invaders, las tres capas del patrón MVC se distribuyen de la siguiente manera:

Modelo

La clase central del modelo es **Espacio**, que está implementado como **Singleton** y extiende de la clase **Observable**. Contiene toda la lógica del juego: la instancia del **Jugador**, la lista de **Enemigos** y el objeto **Disparo** asociado al jugador. Además, **Espacio** alberga el bucle principal del juego (*game loop* con un **Timer** de 50 ms), la detección de colisiones entre el disparo y los enemigos, y las condiciones de fin de partida (*game over* y victoria).

Las clases auxiliares del modelo definen la estructura de las naves y los disparos del juego:

- **Naves** — clase abstracta que encapsula el comportamiento común de todas las naves. Define atributos: **x** (posición horizontal), **y** (posición vertical), **velocidad** (píxeles por actualización) y **vivo** (estado actual). Incluye el método abstracto **mover()**, que cada subclase implementa según su lógica de movimiento.
- **Jugador** — extiende **Naves** y representa la nave controlable. Implementa **mover()** para desplazarse únicamente dentro de los límites del tablero visible. Posee un objeto **Disparo** que se dispara cuando el usuario presiona espacio. Solo hay una instancia del jugador simultáneamente.
- **Enemigo** — extiende **Naves** y representa los enemigos del juego. Se crean constantemente en posiciones aleatorias en la fila superior y descienden verticalmente píxel a píxel. Se eliminan al salir del tablero o ser impactados por un disparo.
- **Disparo** — representa el proyectil disparado por el jugador. Almacena su posición actual, un booleano para saber si está en vuelo, y la posición del jugador al momento del disparo. Ascende un píxel por tick. Se desactiva automáticamente al salir del tablero.

Ninguna de estas clases conoce la interfaz gráfica ni sabe cómo se dibuja el juego; solo gestionan datos y reglas de juego.

Vista

Las vistas son **StartFrame** y **MainFrame**, ambas subclases de **JFrame** e implementan la interfaz **Observer**. Su función es capturar el estado del modelo y representarlo visualmente sin contener lógica de negocio:

- **StartFrame** es la pantalla inicial del juego. Se encarga de mostrar el título, las instrucciones básicas de control, y el mensaje «Pulsa SPACE para jugar». No dibuja ningún elemento del juego ni gestiona la lógica. Su responsabilidad es permitir que el usuario active el inicio de la partida.
- **MainFrame** es la pantalla de juego principal y gestiona toda la representación gráfica. Dibuja el tablero como cuadrícula negra de 100×60 celdas, el jugador como rectángulo magenta, los enemigos como rectángulos rojos, y el disparo como rectángulo amarillo. También muestra el mensaje de victoria o derrota cuando la partida termina.

El mecanismo clave es que ambas vistas se registran automáticamente como observadoras de **Espacio** al construirse, mediante la llamada **Espacio.getEspacio().addObserver(this)** en sus constructores. Esto garantiza que recibirán notificaciones automáticas de cualquier cambio en el estado del juego.

Controlador

Los controladores son clases internas privadas llamadas **Controller** definidas dentro de **StartFrame** y **MainFrame**. Ambas implementan **KeyListener**. En lugar de implementar los tres métodos de la interfaz (**keyPressed**, **keyReleased** y **keyTyped**), solo se sobrescribe **keyPressed**, lo que es más eficiente. Esto permite que el controlador reciba únicamente los eventos de interés, interprete las teclas pulsadas y traduzca en llamadas concretas al modelo:

- El **Controller** de **StartFrame** escucha la pulsación de **SPACE** para llamar a `Espacio.getEspacio().cambiarAMain()`, indicando al modelo que inicie la partida.
- El **Controller** de **MainFrame** atiende las cuatro flechas direccionales (llamando a `espacio.moverJugador(dx, dy)` con los desplazamientos) y la barra espaciadora (llamando a `espacio.disparar()`). Además, verifica si la partida ha terminado y rechaza cualquier entrada en ese caso.

La relación entre las tres capas queda así: el usuario interactúa con la **Vista** (ventana con foco de teclado), el **Controlador** captura esa interacción y llama al **Modelo**, y el Modelo notifica a la Vista para que se hagan los cambios necesarios. En ningún momento la vista accede directamente a la lógica del juego, ni el modelo sabe cómo se representa gráficamente.

Más adelante se verá como se implementa esta relación entre las tres capas en el proyecto Space Invaders.

Diagramas UML

El Controlador en Detalle

El controlador es el intermediario entre el usuario y el sistema. Recibe las acciones del usuario (eventos de teclado), las interpreta según la lógica de control definida, y solicita al modelo que execute las operaciones correspondientes. No contiene datos del juego ni se encarga de mostrarlos; simplemente traduce entrada en acciones del modelo.

Implementación del Controlador en Space Invaders

Clase MainFrame

El **Controller** de esta clase es la parte que traduce las teclas del usuario en acciones del juego. Está definido como una clase interna en **MainFrame** e implementa **KeyListener**, por lo que escucha eventos del teclado.

Su flujo es el siguiente:

1. Se crea en **MainFrame** con `Controller controller = new Controller()`.
2. Se registra en la ventana con `addKeyListener(...)` en **MainFrame**.
3. Para que realmente reciba teclas, el frame solicita el foco en **MainFrame**.

Lo importante ocurre en `keyPressed` en **MainFrame**:

- Primero comprueba si la partida ya terminó: si el juego está en *game over* o *ganado*, no hace nada.
- Luego mira qué tecla se pulsó con `e.getKeyCode()`.
- Según la tecla, llama al modelo **Espacio**:
 - **Flecha izquierda:** mueve al jugador a la izquierda.
 - **Flecha derecha:** mueve al jugador a la derecha.

- **Flecha arriba:** mueve al jugador hacia arriba.
- **Flecha abajo:** mueve al jugador hacia abajo.
- **Espacio:** dispara.

En otras palabras: el **Controller** no dibuja nada ni calcula reglas visuales; solo captura la entrada del teclado y le dice al modelo qué hacer. Después, cuando el modelo cambia y notifica a la vista, el método **update** repinta el panel en **MainFrame**.

Clase **StartFrame**

En esta clase, el **Controller** sirve para controlar la pantalla de inicio usando el teclado. Está definido al final de **StartFrame** como una clase interna que implementa **KeyListener**. Su función es muy simple: escuchar si el usuario pulsa la barra espaciadora. El flujo es el siguiente:

1. En el constructor de **StartFrame** se crea el controller y se registra con **addKeyListener(controller)**.
2. Como la ventana pide el foco con **StartFrame**, puede recibir eventos del teclado.
3. Cuando el usuario pulsa una tecla, se ejecuta **keyPressed** en **StartFrame**.

Dentro de **keyPressed**, el controller hace una sola comprobación:

- Si la tecla pulsada es **SPACE**, llama a **iniciarJuego()**.

Ese método está en **StartFrame** y lo que hace es pedir al modelo **Espacio** que cambie al estado principal del juego con **cambiarAMain()**. Después, como la clase también observa al modelo, el método **update** en **StartFrame** responde a ese cambio:

- Elimina este frame como observer.
- Oculta la ventana de inicio.
- Abre un nuevo **MainFrame**.

En resumen: el **Controller** aquí no mueve naves ni dispara. Solo detecta que el usuario pulsa **SPACE** en la pantalla inicial y lanza el arranque del juego.

Patrón Observer: Notificación y Actualización

Dentro del patrón Observer, los métodos **notify** y **update** permiten que los objetos que observan a otro objeto puedan mantenerse informados cuando su estado cambia.

Método notify

El método **notify** pertenece al objeto **observable**, que es el objeto que contiene los datos o el estado que puede cambiar. En muchas aplicaciones que siguen la arquitectura MVC, el observable suele ser el modelo, ya que es el componente que gestiona la información y la lógica del sistema.

La función del método **notify** es avisar a todos los observadores registrados de que el estado del observable ha cambiado. Para ello, el observable mantiene normalmente una lista o colección de observadores que se han registrado previamente para recibir notificaciones.

Cuando ocurre un cambio relevante en el estado del observable (por ejemplo, una modificación de datos), el método **notify** recorre esta lista de observadores y llama al método **update** de cada uno de ellos. De esta manera, todos los observadores reciben la notificación del cambio y pueden reaccionar en consecuencia.

Una característica importante del método **notify** es que el observable **no necesita conocer los detalles internos de los observadores**. Simplemente se encarga de avisarlos de que ha ocurrido un cambio. Esto permite mantener un bajo acoplamiento entre las clases, lo que facilita la reutilización del código y el mantenimiento del sistema.

Método update

El método **update** pertenece a los **observadores**. Cada observador debe implementar este método para definir cómo reaccionará cuando el observable le notifique un cambio.

Cuando el observable ejecuta el método **notify**, este llama al método **update** de cada observador registrado. En ese momento, el observador recibe la notificación y puede realizar las acciones necesarias para actualizar su estado.

En muchos casos, el método **update** se utiliza para que el observador consulte el nuevo estado del observable y adapte su comportamiento o su representación visual. Por ejemplo, si el observable es el modelo de una aplicación y el observador es una vista, el método **update** se encargará de actualizar la información que se muestra al usuario para reflejar los cambios recientes.

Cada observador puede implementar su propio comportamiento en el método **update**, lo que permite que diferentes componentes reaccionen de distintas formas ante el mismo cambio en el observable.

Funcionamiento conjunto de notify y update

El funcionamiento coordinado de estos dos métodos permite mantener sincronizados los distintos componentes de un sistema que utiliza el patrón Observer. El proceso general es el siguiente:

1. Se produce un cambio en el estado del objeto observable.
2. El observable detecta ese cambio y ejecuta el método **notify**.
3. El método **notify** recorre la lista de observadores registrados.
4. Para cada observador, se llama a su método **update**.
5. Cada observador recibe la notificación y actualiza su estado o su información según corresponda.

Gracias a este mecanismo, los cambios en el sistema se propagan automáticamente a los objetos que dependen de ellos, sin necesidad de que exista una dependencia directa entre todas las clases.

Aplicación al proyecto Space Invaders

4.4.1. Quién notifica y cuándo

En el proyecto, el observable es `Espacio`. Las notificaciones a los observadores se realizan directamente con la secuencia:

```
setChanged();
notifyObservers(new int[] {caso, datos...});
```

La llamada a `setChanged()` es obligatoria antes de `notifyObservers()`: sin ella, la clase `Observable` no propagaría nada. `notifyObservers()` recorre internamente la lista de observers registrados y llama al `update` de cada uno, pasando un array de enteros con información específica sobre el cambio.

Las notificaciones se invocan desde varios puntos del modelo, cada vez que el estado del juego cambia y la pantalla debe reflejarlo:

- **Al mover al jugador** — dentro de `moverJugador()`, tras actualizar la posición.
- **Al disparar** — dentro de `disparar()`, tras crear el nuevo `Disparo`.
- **Al actualizar el disparo** — dentro de `actualizarDisparo()`, en cada tick del *game loop* (cada 50 ms) mientras el proyectil está activo.
- **Al detectar una colisión** — dentro de `comprobarColisiones()`, cuando el disparo alcanza a un enemigo (el enemigo pasa a `vivo = false` y el disparo se desactiva).
- **Al bajar los enemigos** — dentro de `actualizarEnemigos()`, cada 200 ms, tras desplazar todos los enemigos vivos.

Adicionalmente, `cambiarAMain()` llama directamente a `setChanged()` y `notifyObservers(new int[] {9})` para señalar que el juego ha cambiado al estado principal y que `StartFrame` debe reaccionar.

4.4.2. Cómo responde MainFrame en su update

`MainFrame` implementa `Observer` y se registra en `Espacio` al construirse. Su método `update` procesa las notificaciones específicas:

```
@Override
public void update(Observable o, Object arg) {
    if (o instanceof Espacio) {
        int[] datos = (int[]) arg;
        procesarNotificacion(datos);
    }
}
```

El método `procesarNotificacion` analiza el primer elemento del array (el tipo de evento) y actualiza selectivamente las celdas del tablero según el caso: movimiento del jugador, disparo, enemigos, colisiones, etc. Esto es más eficiente que repintar todo el tablero en cada cambio y permite actualizaciones visuales más precisas.

4.4.3. Cómo responde StartFrame en su update

StartFrame también implementa Observer y se registra en Espacio al construirse. Su método update tiene un comportamiento distinto: no dibuja nada, sino que gestiona la transición entre pantallas:

```
@Override
public void update(Observable o, Object arg) {
    if (o instanceof Espacio) {
        Espacio.getEspacio().deleteObserver(this);
        this.setVisible(false);
        new MainFrame();
    }
}
```

Cuando Espacio notifica (lo que ocurre al llamar a cambiarAMain() desde el controlador de StartFrame), este update:

1. Se desregistra como observer con deleteObserver(this), para no recibir más notificaciones futuras del juego.
2. Oculta la ventana de inicio con setVisible(false).
3. Crea un MainFrame, que al construirse se registra automáticamente como el nuevo observer y abre la pantalla de juego.

Así, un único evento de notificación actúa como señal de arranque: hace desaparecer la pantalla de inicio y aparece la pantalla de juego, sin que ninguna de las dos clases necesite conocerse directamente entre sí.

Conclusiones

La implementación del patrón MVC junto con Observer en el proyecto Space Invaders demuestra las ventajas de una arquitectura bien estructurada:

- **Separación clara de responsabilidades:** El modelo (Espacio) gestiona exclusivamente la lógica del juego, las vistas (StartFrame, MainFrame) se encargan únicamente de la presentación, y los controladores traducen la interacción del usuario en acciones del modelo.
- **Bajo acoplamiento:** Las clases no dependen directamente unas de otras. El modelo no conoce las vistas, y las vistas no contienen lógica de negocio.
- **Sincronización automática:** El patrón Observer garantiza que cualquier cambio en el modelo se refleje inmediatamente en todas las vistas registradas, sin necesidad de llamadas explícitas.
- **Facilidad de mantenimiento:** Los cambios en la lógica del juego solo afectan al modelo, los cambios en la interfaz solo afectan a las vistas, y las modificaciones en los controles solo afectan a los controladores.
- **Extensibilidad:** Es fácil añadir nuevas vistas (por ejemplo, una pantalla de puntuaciones) o nuevos tipos de interacción sin modificar el código existente.

Esta arquitectura proporciona una base sólida para el desarrollo de aplicaciones interactivas, facilitando tanto el desarrollo inicial como el mantenimiento y la evolución futura del software.

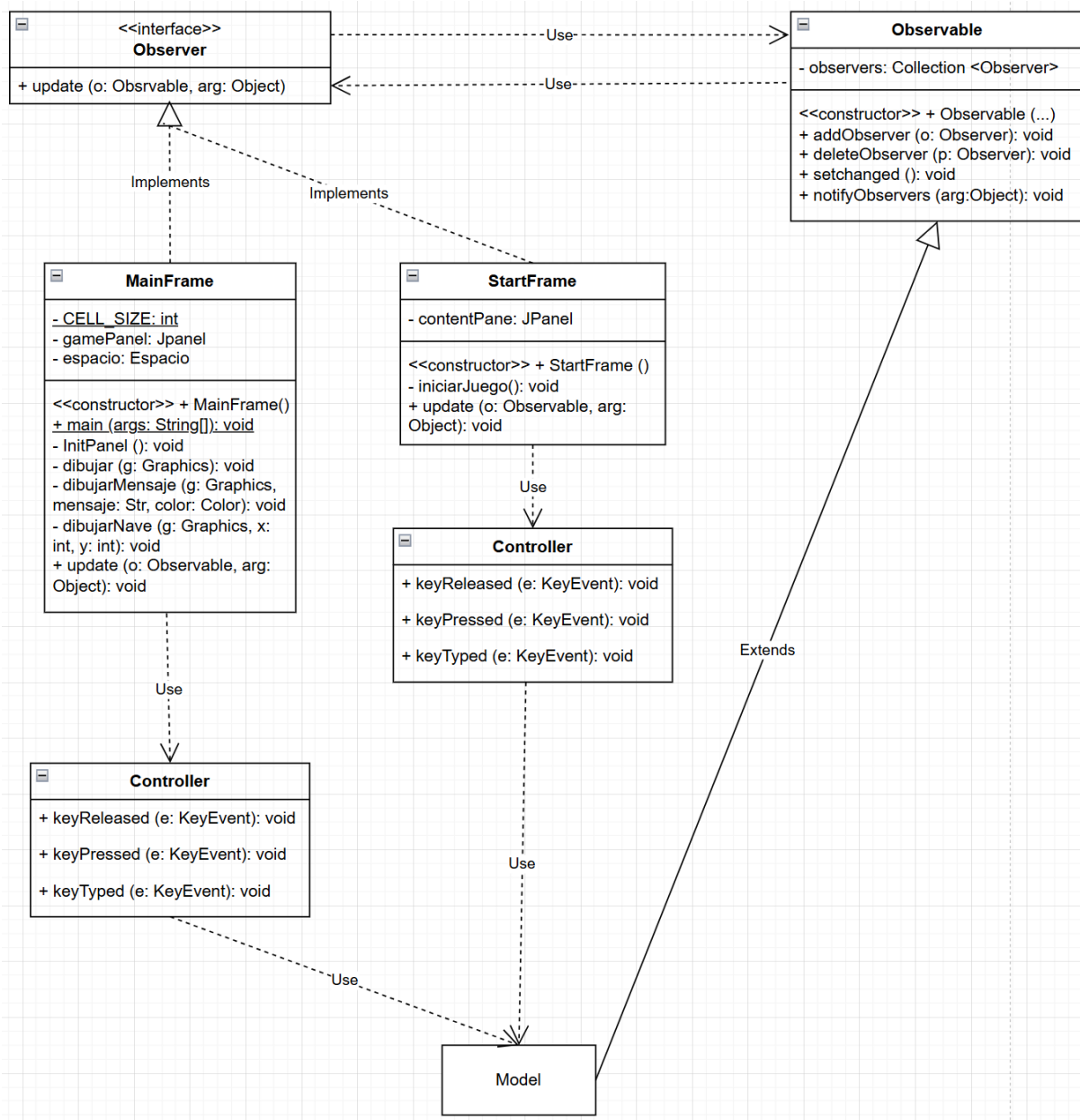


Figura 1: Diagrama UML de la arquitectura MVC y relación Observer entre las clases del proyecto Space Invaders.